Learning Augmentation Mixtures by Matching the Features of Held-Out Transformations

Research draft
Author information to be supplied

Abstract

In corruption-robust continual learning, a classifier learns tasks in sequence and must recognize classes from across that sequence when test images undergo unfamiliar corruptions. MetaMix addresses this problem by adapting how current and replay images are augmented as training progresses. In our MetaMix-guided implementation, the mixer can reduce prediction disagreement by increasing the weight on the original image. This shortcut weakens the signal for choosing among augmentation chains. Our hypothesis is that chains available to the classifier can approximate the feature changes produced by transformations held out from its current update. We use mean squared error (MSE) between the features of a mixed training view and those of independently augmented views of the same image. In the reported batch-32 SmallCNN comparison on Split-CIFAR-10, the feature-matching run reaches 45.05% corrupted class-incremental accuracy. Two learned mixing baselines reach 38.55% and 39.68%, while random mixing reaches 48.27%. The comparison motivates a controlled study of feature matching; differences between controllers limit attribution to the loss itself.

1. Motivation

We study the corruption-robust continual learning problem posed by Wang et al. [3]. A learner receives tasks sequentially, with limited memory for examples from earlier tasks. After learning a new task, it must still recognize previously learned classes, including when test images undergo corruptions unavailable during training. For example, a replay image of a car can help preserve its class after the learner moves on to birds, yet the learner may still misclassify that car under fog. Retention on clean images and accuracy under corruption are therefore distinct evaluation criteria.

MetaMix addresses corruption robustness in continual learning by adapting how current and replay examples are augmented as tasks change [3]. A difficulty arises when the mixer controls augmentation strength as well as chain selection. In our MetaMix-guided implementation, the prediction-consistency objective leaves a shortcut open: the mixer can make its views agree by returning more of the original image, regardless of which chains it selects.

AugMix samples several transformation chains for an image and blends their outputs [2].

A shortcut in the mixer objective. In our MetaMix-guided JS implementation, the updated classifier is held fixed while the mixer minimizes prediction disagreement across the original image and two mixed views. Write a mixed view as $s_v = mx + (1 - m)c_v$, where c_v blends the sampled chains. As $m \to 1$, both views converge to x and their three-way JS divergence converges to zero for *any* choice of chain weights. The mixer can therefore approach the minimum by suppressing augmentation, even if it never learns which chains prepare the classifier for a shift in appearance. In the final epoch of our guided JS run, the mean original-image weight was 0.9726, leaving only 2.74% of the mixture weight on the chains. This observation is consistent with the shortcut in our implementation; the limit argument applies specifically to the direct JS update with a fixed classifier.

We address the two parts of this problem separately. Fixing $m = 0.5$ removes the option of increasing the original-image weight. Feature matching then gives the chain weights a target derived from an independently sampled transformation of the same image.

Following MetaMix [3], we divide the augmentation operations into two pools for each batch. The classifier trains on mixtures from the pseudo-seen pool. The pseudo-unseen pool supplies other views of the same labeled training images. This creates a concrete question for the mixer: which combination of the available seen chains most closely approximates the effect of the held-out transformations on the current representation?

2. Why feature MSE?

A target for choosing chains. Let x be an image, $s_\phi(x)$ a mixture of seen chains with weights produced by controller ϕ, and $u(x)$ an independently sampled unseen chain. The

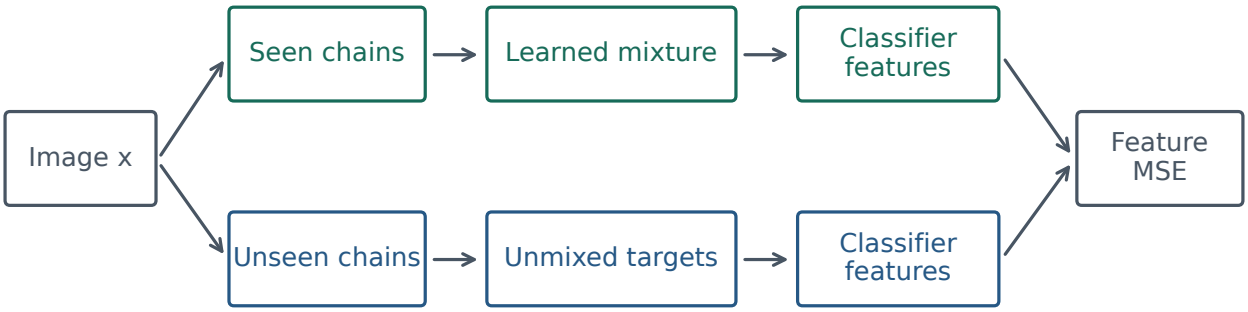


Figure 1. Mixer update for one training image. Both branches use the same frozen classifier. The held-out branch supplies detached targets. Only the learned mixture receives a gradient through its features.

classifier's pooled features h_θ provide a common coordinate system for the two views. Matching features of the same image compares how the two augmentations move its representation. Indeed, the difference between their feature displacements from $h_\theta(x)$ is exactly $h_\theta(s_\phi(x)) - h_\theta(u(x))$. The unseen branch bypasses the controller, so changing the weights changes only the seen mixture.

What minimizing the distance accomplishes. For a fixed image and view, let z denote the seen-mixture feature and t_k the K unseen target features, with mean $\bar{t}$. Squared distance gives the identity

$$\frac{1}{K}\sum_k \|z - t_k\|^2 = \|z - \bar{t}\|^2 + \frac{1}{K}\sum_k \|t_k - \bar{t}\|^2. \quad (1)$$

The second term is constant during the mixer update. The weights therefore push the seen mixture toward the mean held-out representation, subject to what the sampled chains can produce. This gives a precise rationale for MSE and also identifies its limitation: a mean can hide distinct transformation effects. Improving accuracy on real corruptions remains an empirical question.

3. Training protocol

Each batch splits nine AugMix operations into disjoint pools of 5–7 seen and 2–4 unseen operations. For each image we sample two views, each with three chains of depth 1–3 at severity 3. The split changes each batch. CIFAR-10-C images are reserved for evaluation.

Classifier update. Following DER++ [1], the classifier combines current-example cross-entropy (CE), labeled replay CE with coefficient 0.5, and stored-logit replay MSE with coefficient 0.2. Class-aware InfoNCE with coefficient 1 and temperature 0.1 aligns the original and its two seen mixtures. Other views of the same example are positives and different-class examples supply negatives. Stored-logit replay MSE updates the classifier; the feature MSE below updates the mixer.

Mixer update. Following the temporal context design of MetaMix [3], a recurrent controller receives current and replay context. The feature-matching recipes also score each candidate using its feature and prediction changes relative to the original. The controller outputs three softmax weights per image and view. With v indexing the view and a_{vk} the seen chains, the mixed input is

$$s_v(x) = \tfrac{1}{2}x + \tfrac{1}{2}\sum_{k=1}^{3} w_{vk} a_{vk}(x). \quad (2)$$

After the classifier update, its parameters and batch-normalization statistics are fixed. For each image, the mixer averages over two seen mixtures and three detached unseen targets per view:

$$L_{\text{match}}(x) = \frac{1}{6d}\sum_{v=1}^{2}\sum_{k=1}^{3} \left\| h_\theta(s_v(x)) - \text{sg}[h_\theta(u_{vk}(x))] \right\|_2^2, \quad (3)$$

where $d = 128$ for SmallCNN. Three mixer SGD steps reuse the sampled targets. Gradients pass through the fixed classifier to the seen-chain weights. Feature MSE alone trained the mixer in the reported run.

4. Batch-32 comparison

The comparison uses the SmallCNN recipes for five sequential two-class CIFAR-10 tasks. The reference protocol uses ten epochs per task, current batches of 32 and a replay reservoir of 500. Evaluation covers 10,000 clean images and 750,000 corrupted images (15 corruptions at five severities). Class-IL predicts among ten classes. Task-IL supplies the true class pair. Table 1 compares recipes with class-aware InfoNCE for classifier consistency.

The category breakdown in Table 2 shows the same ordering across noise, blur, weather, and digital corruptions:

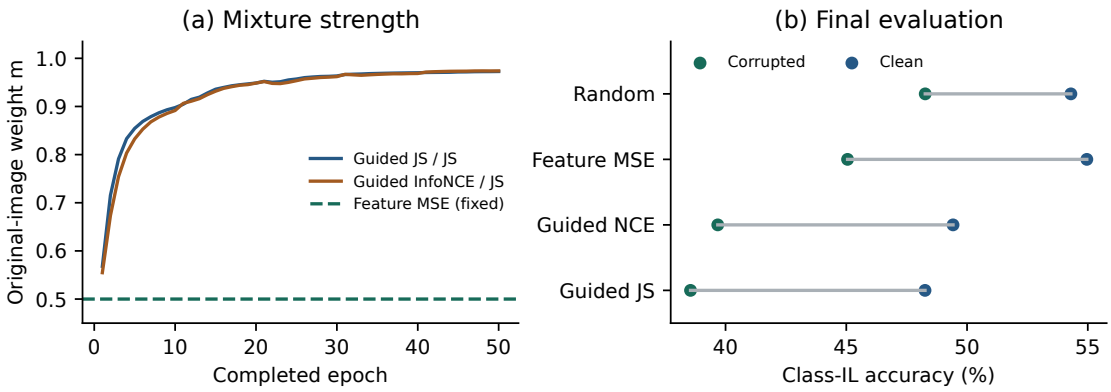


Figure 2. Left: the original-image weight rises during the guided runs; the MSE recipe fixes it at 0.5. Right: among recipes using classifier InfoNCE, feature MSE leads on clean accuracy while random mixing leads on CIFAR-10-C. Guided labels on the right identify the mixer objective.

Mixer	Mixer objective	Clean class-IL	Corrupt class-IL	Clean task-IL	Corrupt task-IL
Random	None	54.30	48.27	91.20	87.19
MetaMix-guided	InfoNCE	49.42	39.68	90.08	83.24
MetaMix-guided	JS	48.26	38.55	89.25	82.69
Feature matching	Feature MSE*	54.96	45.05	90.17	84.72

Table 1. Final accuracy (%) for the reported batch-32 comparison. The imported feature-matching evaluation contains scores only. Its batch size is attributed from the run history. *The experimenter confirmed feature MSE was the sole mixer objective.

Mixer	Noise	Blur	Weather	Digital	All
Random	48.78	49.96	47.48	46.98	48.27
Guided, InfoNCE	36.42	40.35	40.54	40.59	39.68
Guided, JS	34.71	40.06	39.16	39.31	38.55
Feature MSE*	42.02	46.40	44.95	46.08	45.05

Table 2. CIFAR-10-C class-IL accuracy (%) for the batch-32 comparison. Each corruption is averaged over five severities. All weights the 15 corruptions equally. *Feature MSE alone trained the mixer.

feature matching exceeds both guided baselines, while random mixing leads each family.

Comparison with learned mixing. Feature matching reaches 45.05% corrupted class-IL, 6.50 percentage points above guided JS and 5.37 points above guided InfoNCE. It also reaches the highest clean class-IL of the four recipes (54.96%). This pattern supports investigating held-out feature targets as an alternative source of supervision for learned weights. The guided baselines also differ in controller inputs, clean-image weight, and weight sharing across views. The observed differences are recipe-level comparisons.

Comparison with random mixing. Random mixing reaches 48.27% corrupted class-IL, 3.22 points above feature matching, despite scoring 0.66 points lower on clean class-IL. Random mixing also leads on corrupted task-IL. Thus the clean ranking alone would favor a different method than the corruption ranking. The feature-matching run improves over the two learned baselines but leaves a gap to random mixtures.

5. Discussion

The motivation for MSE is strongest when the desired training signal is proximity to the effects of held-out augmentations in the current classifier's feature space. A classifier that already suppresses a transformation supplies little signal for matching it. A classifier that encodes irrelevant changes can reward matching those changes. Neither case establishes how the next classifier update will affect robustness.

To isolate the contribution of MSE, a paired experiment should hold the controller, original-image weight, sampled chains, replay policy, and update budget fixed and vary only the mixer objective (seen-view JS, feature MSE, or their sum). Repeating that comparison across seeds would distinguish a reproducible effect from run variation. The current evidence identifies feature matching as a plausible direction for learned mixing and random mixing as the robustness baseline it must surpass.

References

[1] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for

general continual learning: a strong, simple baseline. In *Advances in Neural Information Processing Systems*, 2020.

[2] Dan Hendrycks, Norman Mu, Ekin D. Cubuk, Barret Zoph, Justin Gilmer, and Balaji Lakshminarayanan. AugMix: A simple data processing method to improve robustness and uncertainty. In *International Conference on Learning Representations*, 2020.

[3] Zhenyi Wang, Li Shen, Donglin Zhan, Qiuling Suo, Yanjun Zhu, Tiehang Duan, and Mingchen Gao. MetaMix: Towards corruption-robust continual learning with temporally self-adaptive data transformation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24521–24531, 2023.